

OpenShift GitOps



7 sections in this chapter:

Section 5.1: GitOps for Kubernetes

Section 5.2: Quiz

Section 5.3: GitOps for Cluster Administration

Section 5.4: Guided Exercise

Section 5.5: GitOps for Application Management

Section 5.6: Guided Exercise

Section 5.7: Lab

GitOps for Kubernetes

Chapter 5 Section 1



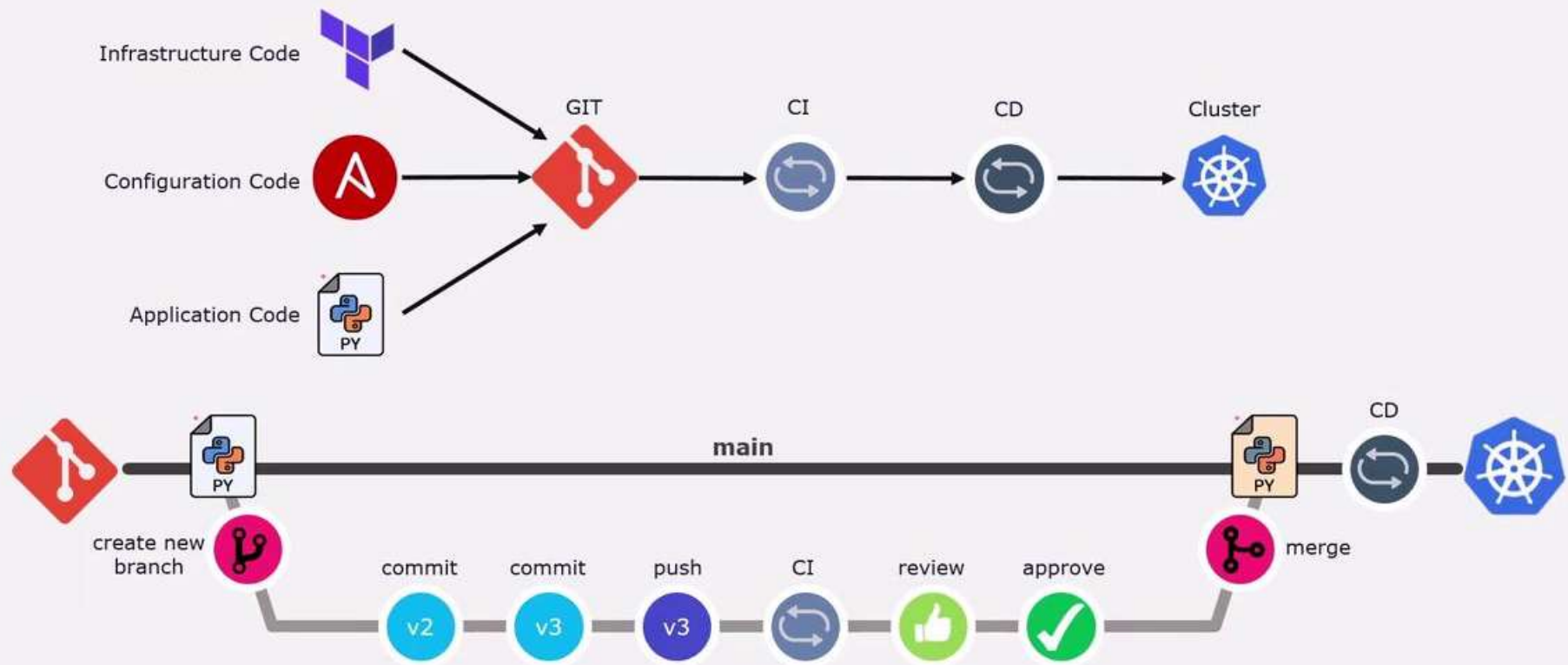
Objectives

After completing this section, you will be:

- Define the
 - fundamentals of GitOps.
- Define its use
 - with Kubernetes clusters and applications.

Introduction to GitOps

GitOps can be considered an extension of Infrastructure as Code (IaC) that uses Git as the version control system



© Copyright KodeKloud

DevOps (CI/CD) practices

Continuous integration (CI)

- integrating changes by trigger automated image built operation.
- automatically run tests and merging code changes.

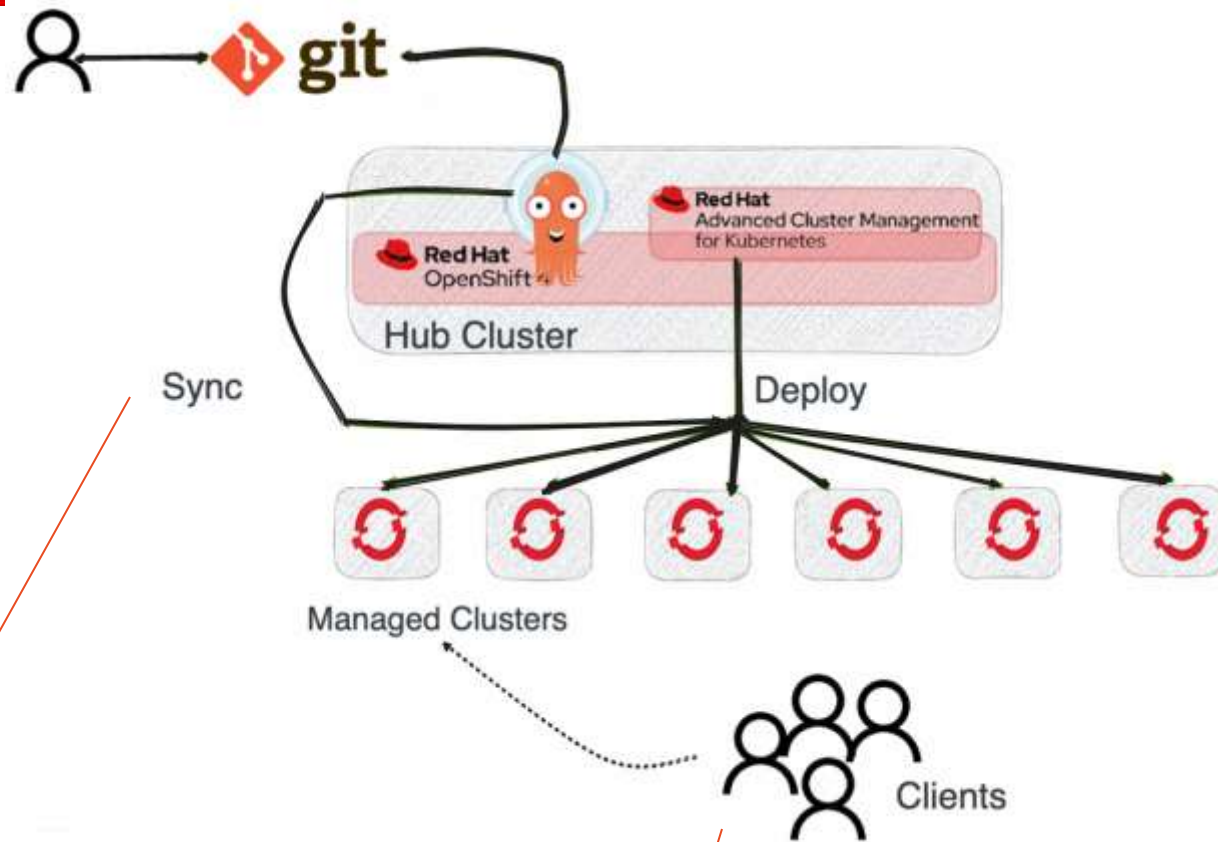
Continuous delivery (CD)

- automates the release to a testing or staging environment
- When changes has been approved then proceed next CD.

Continuous deployment (CD)

- automatically deploys every validated change to production without manual intervention.
- maximizes the release frequency of the project and shortens feedback loops.

Why GitOps?



Git-Centric
Control

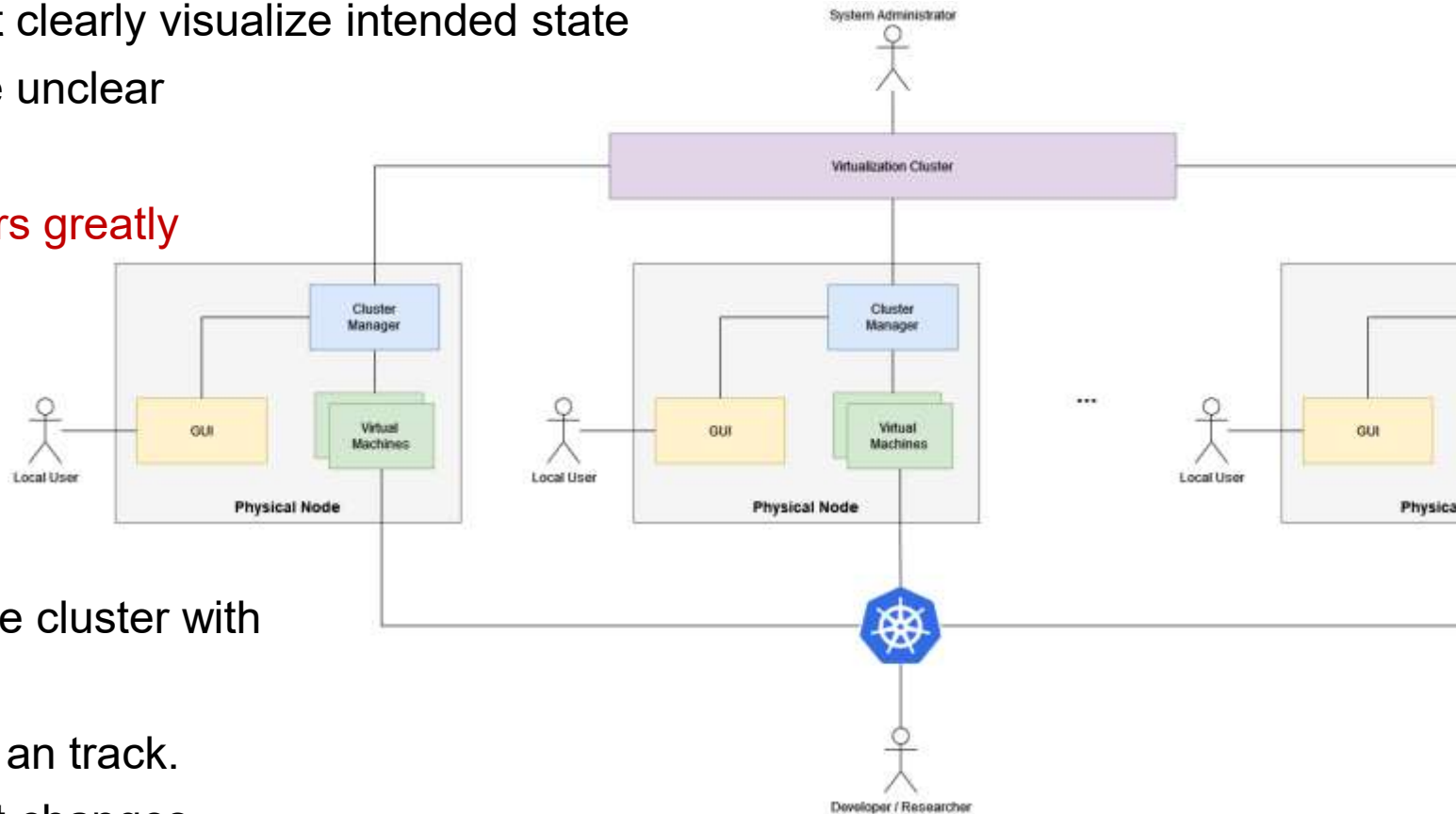
Automated
Reconciliation

Declarative
Desired State

Cluster Management Challenges

Without GitOps

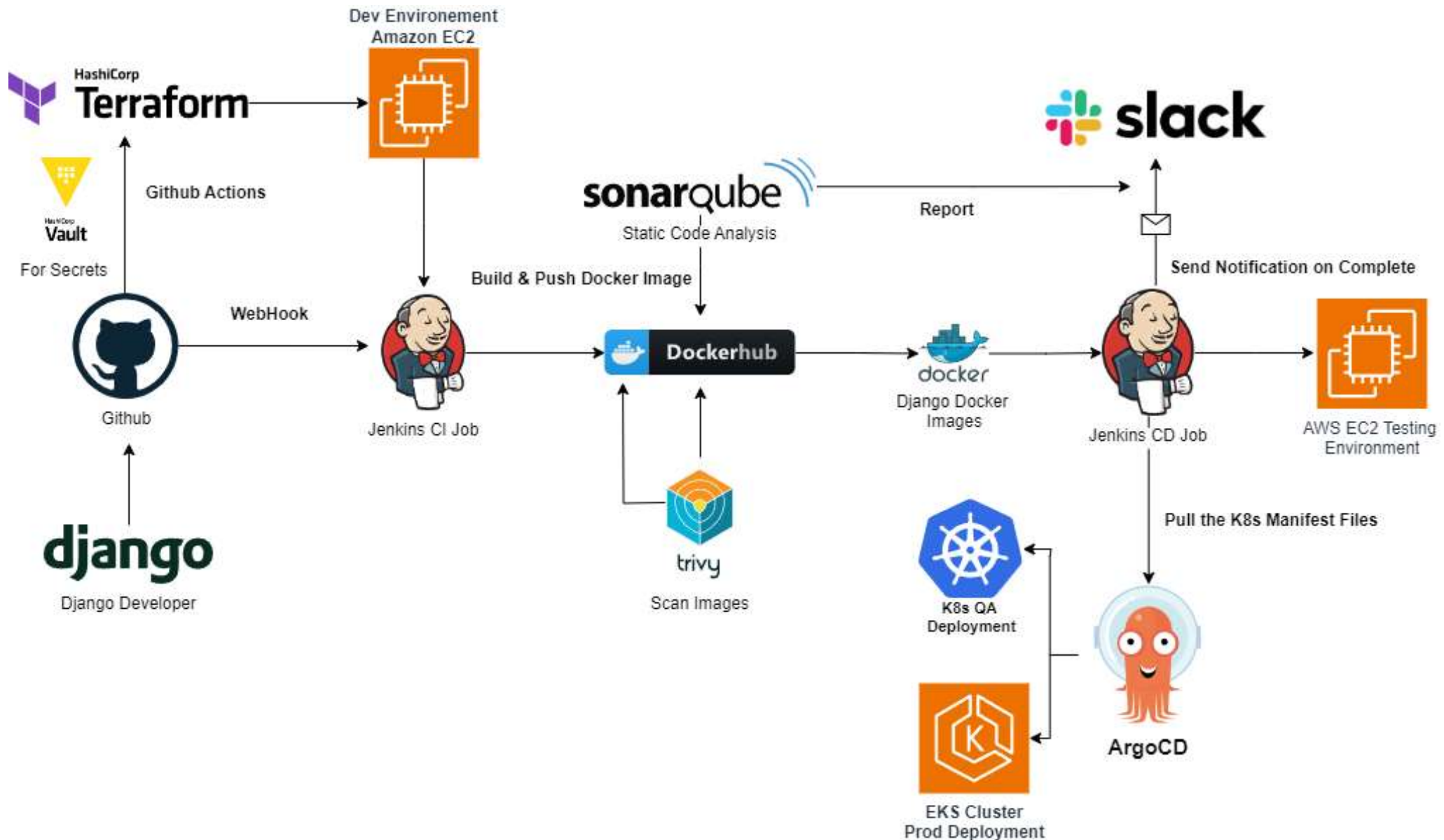
- Multiple users or processes update cluster resources
- Cluster admin cannot clearly visualize intended state
- Cluster state become unclear
- Cluster drift away
 - Intended state differs greatly from desired state.



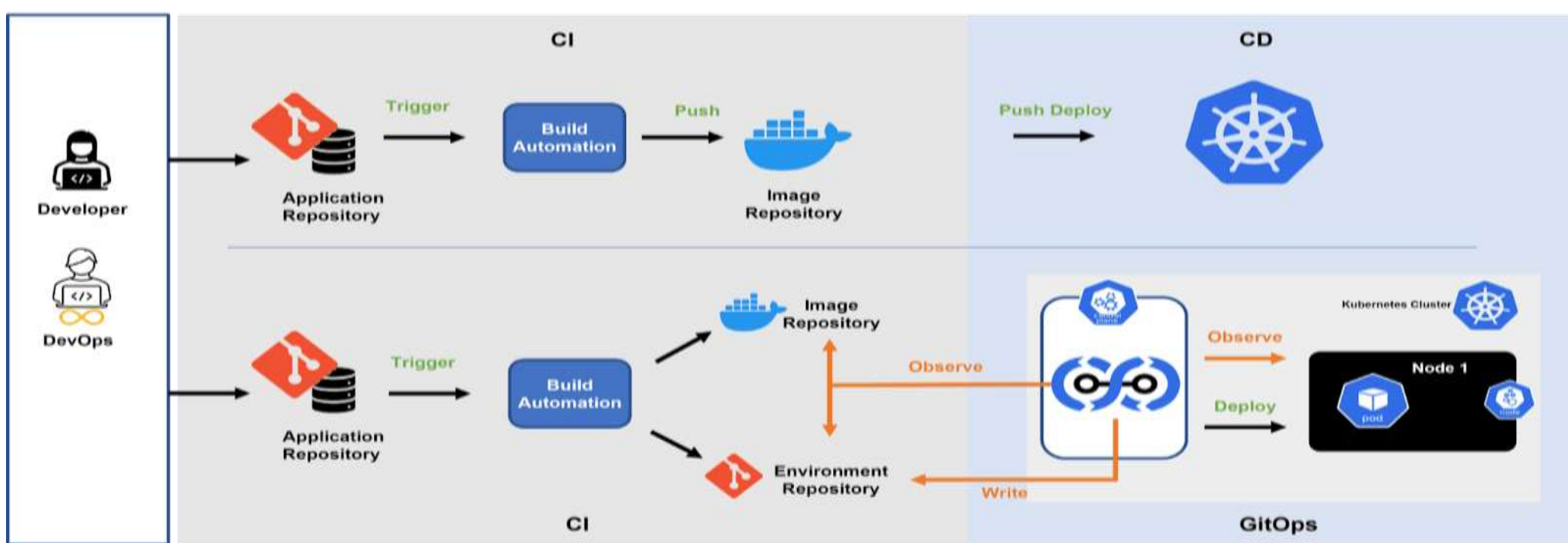
With GitOps

- Single process update cluster with intended definition.
- Admin can reference an track.
- Documentation about changes.

End-to-End CI/CD DevOps

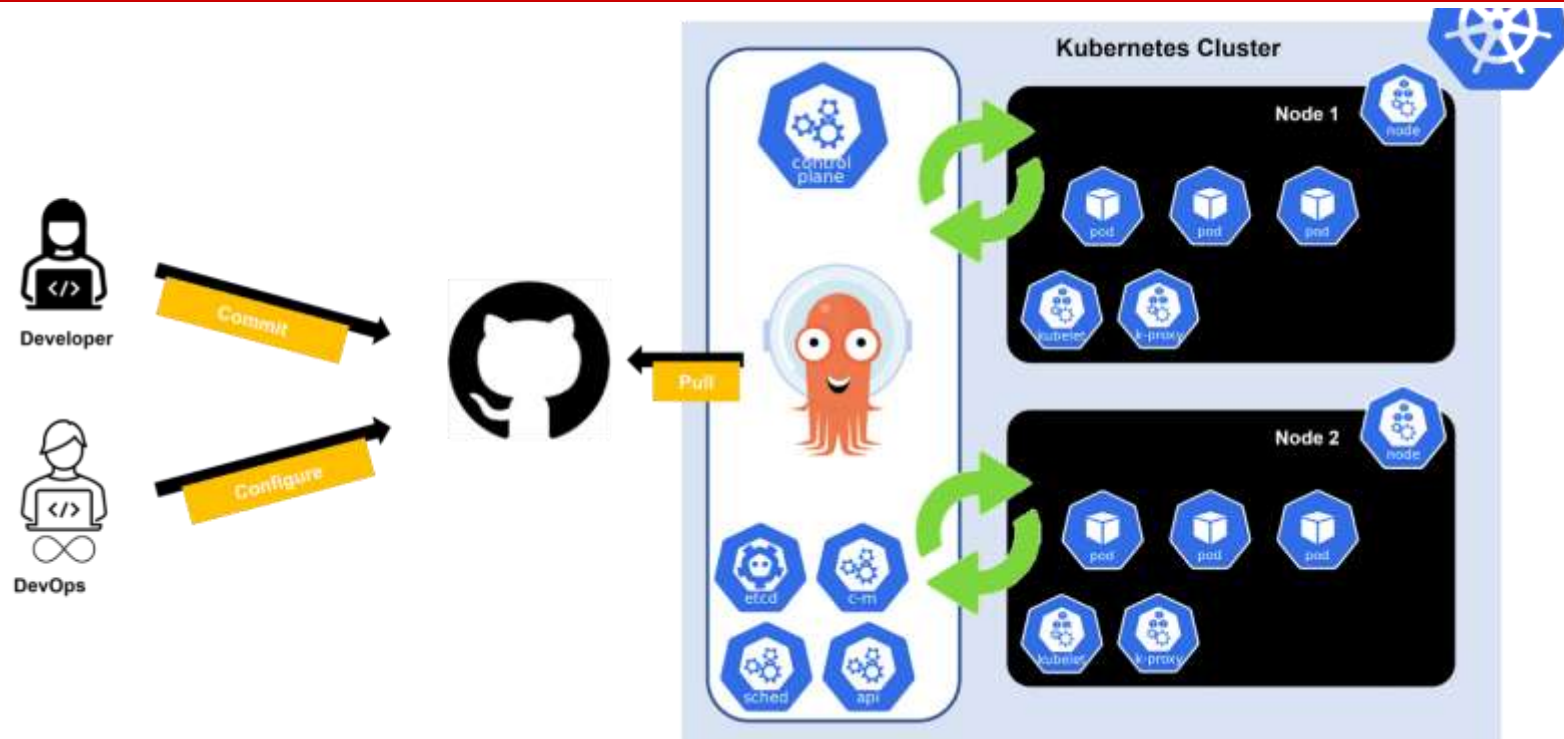


ArgoCD



- GitOps – Pull latest code and configuration data from Git commit, continuous deploy application to cluster.
- Help manage infrastructure – namespace, pod, deployment, replicationset, routes, services.

Best Practices



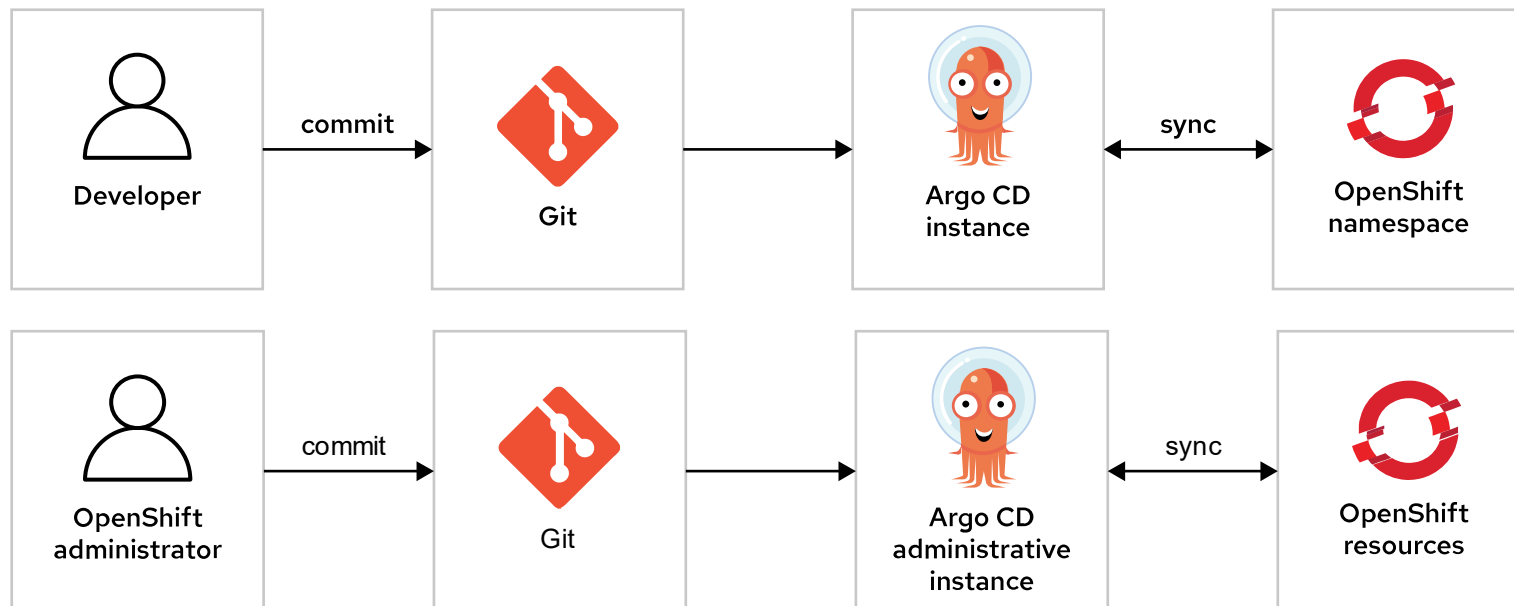
- Separate git repositories for application source code and application data.
- Separate git repositories for system configurations.

Benefits of using ArgoCD

- K8s configurations can be defined as code in the git repository.
- Config files are not applied for individual laptops of developers / DevOps.
- Updates are traceable as tags, branches, or pinned specific manifest versions at Git commits.
- Same and only interface for updating the cluster.
- Git as the single source of truth.
- Avoid untrackable kubectl command applications.
- Version-controlled changes with audit trail.
- Single sign-on (SSO) with providers such as GitLab, GitHub, Microsoft, OAuth2, OIDC, LinkedIn, LDAP, and SAML 2.0.
- Support for webhooks triggering actions in GitLab, GitHub, and BitBucket.
- Easy rollbacks.
- Recreate clusters from scratch.

Introduction to OpenShift GitOps

- Is an operator to
 - install and manages ArgoCD in OpenShift clusters.
 - Install OpenShift Pipelines:
 - run processes when changes occur to git repo.
 - synchronize K8s resources to definitions stored in git repositories.
- Has web console with dashboard showing install plans, operator workloads and other related resources.



Workflows in OpenShift GitOps

1. A developer updates the source code of an application.
2. The developer pushes the change to a Git repository branch.
3. Trigger a pull request to OpenShift Pipelines.
4. OpenShift Pipelines automatically performs :
 - Checks for errors or incorrect formatting in codes.
 - Post update to Git repo – indicate pull request passed checks or contain errors.
 - Trigger ArgoCD job – deploy application with specified changes to testing environment.
5. Team member can review pull request and validate changes.
 - If validation failed, team member can fix issues and re-submit.
6. Team member approve changes
7. OpenShift Pipelines detects approved changes and
 - Merge changes and then build container image from source code.
 - Triggers ArgoCD to synchronize code to production environment.
8. ArgoCD deploys updated image.

Other features of ArgoCD

- Can be deployed as multiple ArgoCD instances for the high availability, permission control, and performance purposes.
- Propagate changes through multiple static environments, such as testing, pre-production, and production environments.
- Limit synchronizations to specific days of the week or periods during the day.
- Add hooks for further processes, such as for sending notifications, pausing monitoring, or triggering backups before an ArgoCD synchronization.

QUIZ: GitOps for Kubernetes

5 questions

Quiz – 1 of 5

What is the purpose of GitOps in the context of Kubernetes?

- a) Deploying changes to production automatically after reviewing and testing them
- b) Storing Kubernetes resource definitions in Git repositories
- c) Improving the tools and diagnostics that engineers use to resolve issues
- d) Using Kubernetes and cloud providers

Quiz – 1 of 5

What is the purpose of GitOps in the context of Kubernetes?

- a) Deploying changes to production automatically after reviewing and testing them
- b) Storing Kubernetes resource definitions in Git repositories**
- c) Improving the tools and diagnostics that engineers use to resolve issues
- d) Using Kubernetes and cloud providers

Quiz – 2 of 5

What are two direct benefits of GitOps? (Choose two.)

- a) Applying techniques such as version control to Kubernetes cluster administration
- b) Automated testing for infrastructure changes
- c) Aligning objectives between the development and operations teams
- d) Improving control over cluster changes, helping to audit changes, and using tools for approvals and reviews

Quiz – 2 of 5

What are two direct benefits of GitOps? (Choose two.)

- a) Applying techniques such as version control to Kubernetes cluster administration**
- b) Automated testing for infrastructure changes
- c) Aligning objectives between the development and operations teams**
- d) Improving control over cluster changes, helping to audit changes, and using tools for approvals and reviews

Quiz – 3 of 5

How is Red Hat OpenShift GitOps delivered?

- a) A Helm chart
- b) Kustomization
- c) An operator
- d) A base OpenShift component

Quiz – 3 of 5

How is Red Hat OpenShift GitOps delivered?

- a) A Helm chart
- b) Kustomization
- c) **An operator**
- d) A base OpenShift component

Quiz – 4 of 5

In which two ways does OpenShift GitOps help with controlling changes to a cluster? (Choose two.)

- a) OpenShift GitOps manages permissions of Git repositories, to limit who can push changes.
- b) OpenShift GitOps helps administrators limit access to the Kubernetes API, because changes can be submitted through Git.
- c) With OpenShift GitOps, administrators can create multiple ArgoCD instances to improve control over permissions.
- d) OpenShift GitOps validates changes to the cluster, so administrators can implement policies to prevent certain changes.

Quiz – 4 of 5

In which two ways does OpenShift GitOps help with controlling changes to a cluster? (Choose two.)

- a) OpenShift GitOps manages permissions of Git repositories, to limit who can push changes.
- b) OpenShift GitOps helps administrators limit access to the Kubernetes API, because changes can be submitted through Git.**
- c) With OpenShift GitOps, administrators can create multiple ArgoCD instances to improve control over permissions.**
- d) OpenShift GitOps validates changes to the cluster, so administrators can implement policies to prevent certain changes.

Quiz – 5 of 5

Which practice does OpenShift GitOps relate to?

- a) DevOps
- b) Agile
- c) Continuous delivery and deployment
- d) Public cloud

Quiz – 5 of 5

Which practice does OpenShift GitOps relate to?

- a) DevOps
- b) Agile
- c) Continuous delivery and deployment**
- d) Public cloud

Section 5.2 summary



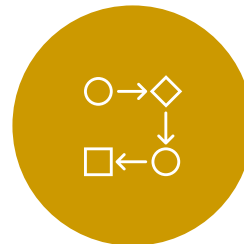
About GitOps



Integrating GitOps with K8s



Install and Use OpenShift GitOps



GitOps Workflow

GitOps for Cluster Administration

Chapter 5 Section 3



Objectives

After completing this section, you will be:

- Configure the default ArgoCD instance from
 - **OpenShift GitOps for cluster administration.**

Red Hat OpenShift GitOps Operator

- Installed by:
 - Using OperatorHub (web UI)
 - Using oc command line
- Steps to install Operator via CLI:
 - Create a openshift-gitops-operator namespace.
 - Create a OperatorGroup object YAML file:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-gitops-operator
  namespace: openshift-gitops-operator
spec:
  upgradeStrategy: Default
```
 - Apply the OperatorGroup to cluster
 - Create and apply subscription to Red Hat OpenShift GitOps Operator

ArgoCD custom resources



- ArgoCD custom resource definition:
 - Installed by OpenShift GitOps operator.
 - Can be used to create instances in cluster.
- Multiple instances with different permissions to several cluster or project.

The Default ArgoCD Instance

- Created
 - by OpenShift GitOps Operator.
 - In openshift-gitops namespace.
 - With route to access the ArgoCD web console.
- Has limited configuration.
- Has sufficient permission to support cluster administration.
- Red Hat recommends:
 - Using this instance for administering cluster
 - Or create more instances for regular applications.

Setting the Default ArgoCD Instance as unmanaged

- Operator manages the default instance
- Any modification to instance, operator can restore changes.
- To prevent operator from restoring changes to the default instance.
 - Remove the whole `ownerReferences` key.
- Consider ignore default instance.
 - Always create own unmanage instance.

```
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  creationTimestamp: "2024-01-02T11:27:35Z"
  finalizers:
    - argoproj.io/finalizer
  generation: 1
  name: openshift-gitops
  namespace: openshift-gitops
  ownerReferences:
    - apiVersion: pipelines.openshift.io/v1alpha1
      blockOwnerDeletion: true
      controller: true
      kind: GitopsService
      name: cluster
      uid: a7a0a073-0328-449e-8ffc-038854feb1c9
  resourceVersion: "262080"
  uid: 1d530e04-8068-4f6f-a066-c3ae36ba34f4
spec:
  applicationSet:
...output omitted...
```

Configuring ArgoCD RBAC

- Interact with ArgoCD thru:
 - Kubernetes API.
 - ArgoCD API
 - Argo built-in web UI
 - Command-line tool
- Authentication to ArgoCD
 - Use local user.
 - Use external IdP.
 - Integrate with OpenShift cluster authentication.

ArgoCD authentication and RBAC configuration

sso:

dex:

resources:

limits:

cpu: 500m

memory: 256Mi

requests:

...output omitted...

openShiftOAuth: true

provider: dex *...output omitted...*

rbac:

defaultPolicy: ""

policy: |

g, system:cluster-admins, role:admin

g, cluster-admins, role:admin

scopes: '[groups]'

...output omitted...

Configuring Trusted Certificates for Git Repository Access

- The repo section key handles communicating with Git repositories.
- Change trusted certificate authorities by
 - Alter the mountPath point to different cert bundle

repo:

volumeMounts:

- mountPath: /etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem
 - name: *volume_name*
 - subPath: *bundle_name*

volumes:

- configMap:
 - name: ***configuration_map_name***
 - name: *volume_name*

Configure default instance to trust cluster CA

- Create cluster root ca bundle configuration map in openshift-gitops namespace.

```
$ oc create configmap -n openshift-gitops cluster-root-ca-bundle
```

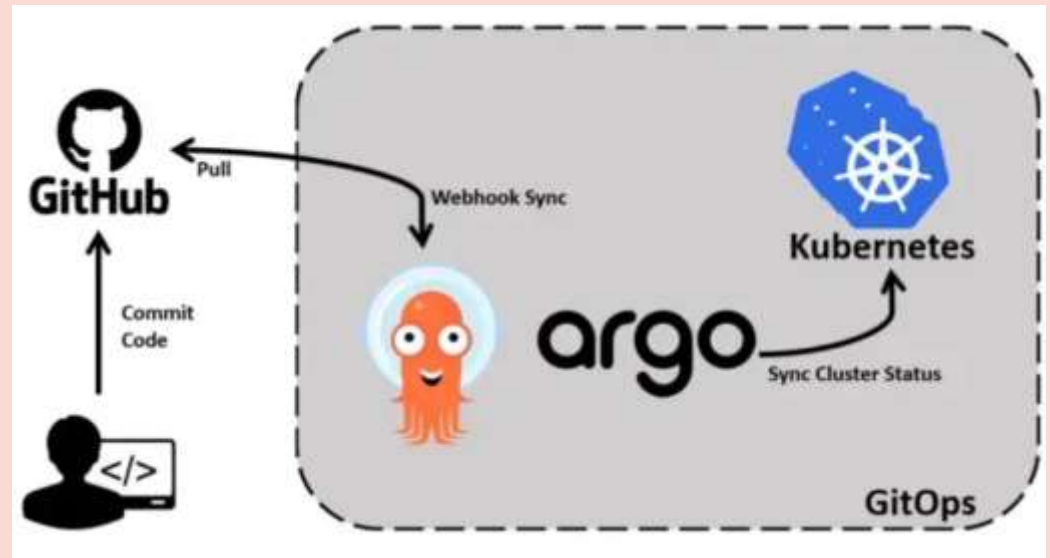
- Add appropriate label to the config map

```
$ oc label configmap -n openshift-gitops cluster-root-ca-bundle \
  config.openshift.io/inject-trusted-cabundle=true
```

Enable route with TLS termination

- Default ArgoCD instance has self-signed HTTPS cert.
- Default ArgoCD doesn't enable route for external access to the web UI.
- Set route termination to reencrypt:
 - Ensures end-to-end encryption.

```
apiVersion: argoproj.io/v1alpha1
kind: ArgoCD
metadata:
  ...output omitted...
spec:
  ...output omitted...
server:
  ...output omitted...
route:
  enabled: true
  tls:
    termination: reencrypt
  ...output omitted...
```

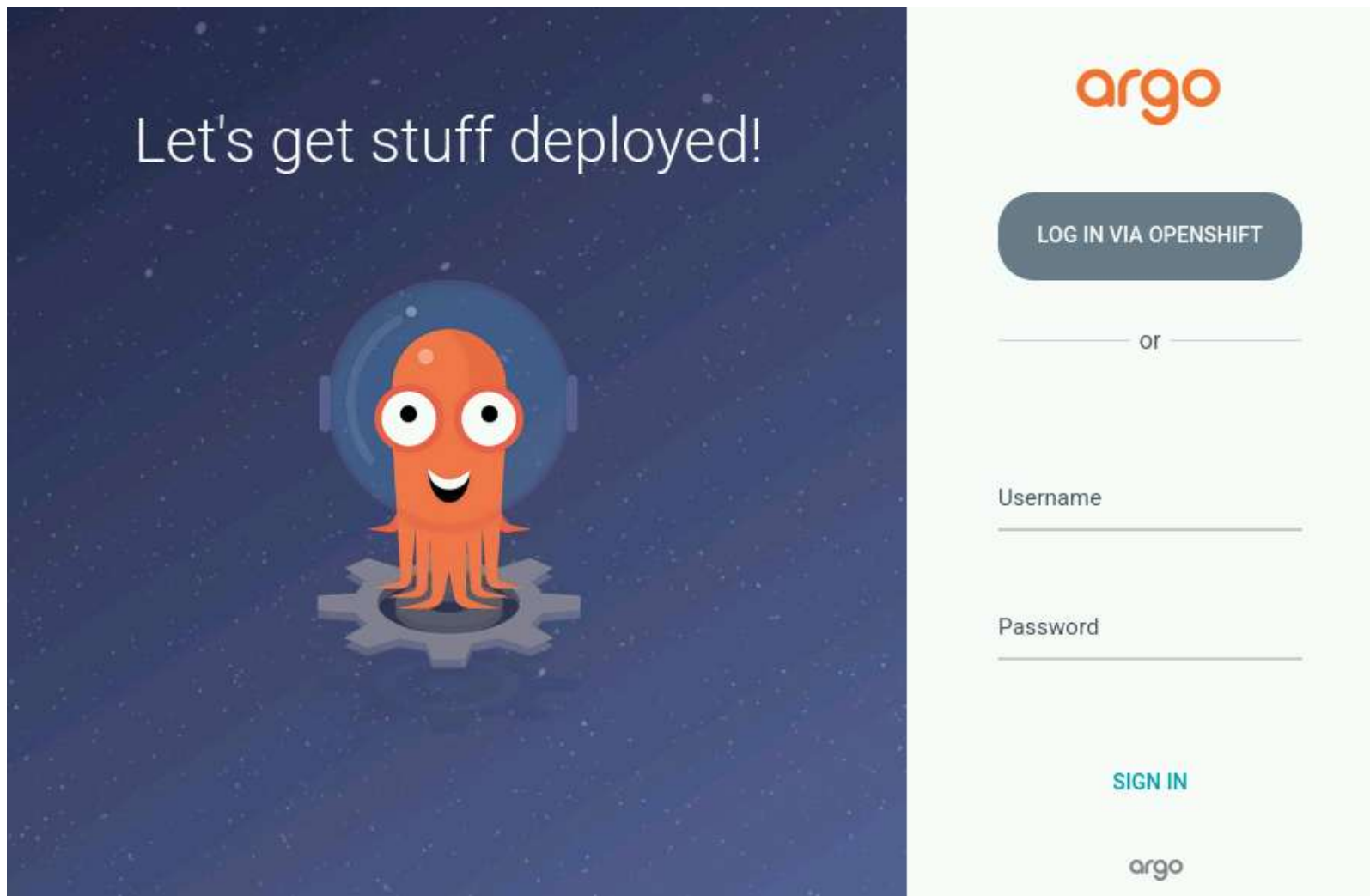


The ArgoCD Web Interface

- Get the route hostname:

```
$ oc get route -n argocd-namespace
```

```
argocd-server argocd-server-gitops-app.apps.ocp4.example.com ...
```



The navigation pane

- Applications: **view** and create applications
- Settings: **change** authentication method, change user password
- User Info: **view** details about logged-in user.

The screenshot displays the Argo CD web interface. On the left is a dark navigation pane with the Argo logo and version 'v2.8.4+ebab8ec'. It contains four main menu items: 'Applications' (with a cube icon), 'Settings' (with a gear icon), 'User Info' (with a person icon), and 'Documentation' (with a book icon). Below these is a 'Favorites Only' toggle (checkbox and star icon) and a 'SYNC STATUS' section with three filterable items: 'Unknown' (0), 'Synced' (1), and 'OutOfSync' (0). The main content area is titled 'Applications' and features three buttons: '+ NEW APP', 'SYNC APPS', and 'REFRESH APPS', along with a search bar. A modal window for an application named 'example' is open, showing its details: Project 'default', Label '...', Status 'Healthy Synced' (with heart and checkmark icons), Repository 'https://github.com/argoproj/...', Target 'HEAD', Path '...', Destination 'in-cluster', Name '...', and Creation time '01/02/2024 08:01:07 (7 min...)'. At the bottom of the modal are three buttons: 'SYNC', 'Refresh', and 'Close'.

Creating Applications

- Use forms to customize the application.
- ArgoCD web console also provides YAML resource editor.

CREATE

CANCEL

×

GENERAL

EDIT AS YAML

Application Name

Project Name

SYNC POLICY

Manual

▼

Create application form field - explained

Application Name The application resource name.

Project Name ArgoCD uses *projects* to group applications and to apply further access controls. Use default project that ArgoCD provides.

Sync Policy The *Automatic* policy synchronizes periodically. With the *Manual* policy, administrators must trigger synchronizations.

Retry add retries to the synchronization process. You might need to configure retries when working with complex applications.

Repository URL point to application source code location on git server.

Path The path in the Git repository that contains the resource definitions. Uses “.” for root path in repo URL.

Cluster URL The URL of the Kubernetes API for the cluster to deploy the application to. Always use `https://kubernetes.default.svc`

Application workflows in details

- Detail page display all Kubernetes resources associated with ArgoCD application.
- Click each resource to get more information such as synchronization status and other details.

The screenshot shows the ArgoCD 'APPLICATION DETAIL' page for an application named 'example'. At the top, there's a search bar with 'example' and a navigation bar with buttons: APP DETAILS, APP DIFF, SYNC, SYNC STATUS, HISTORY AND ROLLBACK, DELETE, and REFRESH. Below the navigation bar, there are three tabs: 'Healthy', 'Syncing', and 'Failed'. The 'Failed' tab is selected, showing a sync failure message: 'Failed a few seconds ago (Tue Jan 02 2024 08:12:43 GMT-0500)'. The failure details include the author 'Anand Francis Joseph' and a comment 'Template variable for container port (#251)'. Below the failure message, there's a diagram showing the application's resources. The application 'example' is shown with a status of 'Syncing' and a sync time of '12 minutes'. It has four resources: 'helm-hooks' (ns), 'kustomize-guestbook' (ns), 'sync-waves' (ns), and 'helm-guestbook' (application). Each resource has a status icon (green checkmark for success, red X for failure) and a sync time (e.g., 'a few seconds').

Section 5.3 Summary



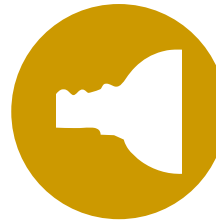
About the ArgoCD default instance



Configure ArgoCD Custom Resources



Configure ArgoCD RBAC



Configure Trusted Certificates for git repo access



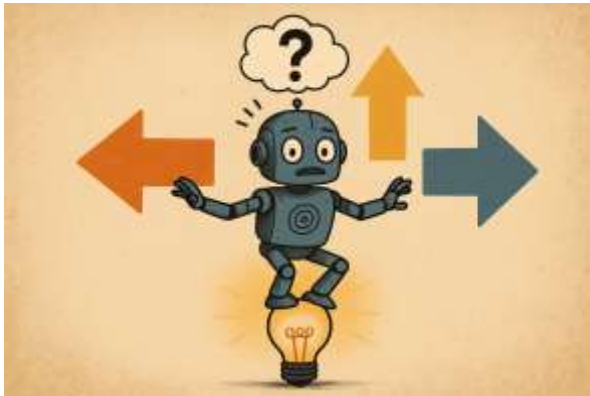
Configure Cluster CA



Use ArgoCD web UI



**Continue or
Guided Exercises or
Break?**



Section 5.4 : Guided Exercise GitOps for Cluster Administration

You should be able to:

- Install Red Hat OpenShift GitOps.
- Configure the default ArgoCD instance.
- Create an ArgoCD application that deploys an operator and custom resources from the operator.
- Patch a cluster resource by using server-side apply.

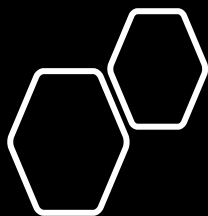
Estimated completion time: 25-45 minutes

Section 5.6 : Guided Exercise GitOps for Application Management

You should be able to:

- Create an ArgoCD instance for developers with the appropriate permissions.
- Use ArgoCD to deploy an application in the cluster.

Estimated completion time: 15 - 25 minutes



Lab: OpenShift GitOps

You should be able to use the OpenShift web console to:

- Define the fundamentals of GitOps and its use with Kubernetes clusters and applications.
- Deploy a GitOps instance for cluster administration.
- Deploy another GitOps instance for application developers and administrators.

Estimated completion time: 45 – 60 minutes

Chapter summary



GitOps



Red Hat OpenShift
GitOps Operator



Install, manage and
operate ArgoCD